

Viewer delle popolazioni sintetiche GSP — documento di design

v0.1 — 29 luglio 2026 — nessun codice, solo design Riferimento dati:

`GSP_popolazioni_full_riferimento_v16.md` (§ citati sotto).

0. Nome

Cinque candidati, con la ragione per cui funzionano.

nome	perché	rischio
Stato delle Anime	è il nome storico del censimento parrocchiale italiano: il registro delle anime di una comunità. Qui le anime sono sintetiche. Colto, italiano, memorabile, e regala tutta l'estetica (registro, timbro, colonne)	«anime» da solo collide con l'animazione giapponese nelle ricerche; va usato per esteso
Microstato	fisica-nativo: la popolazione è <i>un</i> microstato compatibile con i vincoli macroscopici. Coerente col framing MaxEnt del workshop di settembre	opaco per un pubblico non fisico
Sosia	ogni individuo è il doppio statistico di nessuno in particolare. Breve, italiano, un po' inquietante nel modo giusto	poco descrittivo
Atlante GSP	sobrio, funziona in inglese (<i>GSP Atlas</i>), zero ambiguità	anonimo
Specula	latino: specchio e torre d'osservazione insieme	criptico

Raccomandazione: **Stato delle Anime** come nome pubblico (`stato-delle-anime`) come repo e come URL), **Microstato** come titolo alternativo se la vetrina primaria diventa il pubblico dei fisici. Il nome pubblico e il nome del pacchetto Python possono divergere senza costi: `gsp_viewer/` resta il nome tecnico.

1. Scopo, utenti, non-obiettivi

Tre utenti, in ordine di priorità temporale.

- Tu, adesso.** Ispezione, controllo qualità, ricerca di anomalie prima che finiscano in un paper. È l'unico utente per cui la velocità di iterazione conta più della grafica.
- I collaboratori** (Tarantino, Pachet, Zucker, il consorzio PRISM). Devono capire cosa *possono chiedere ai dati* senza leggere 1.000 righe di documentazione. Link permanenti citabili in un paper.

3. **Il pubblico** — amministrazioni, giornalisti, cittadini. Solo in una fase successiva, e con vincoli specifici (§7.3).

Non-obiettivi dichiarati, da scrivere nella pagina Metodo:

- non è uno strumento di stima locale: nessun numero di questa app è una misura del quartiere reale;
- non genera popolazioni: consuma i `_full` già prodotti;
- non fa inferenza causale né previsione.

2. Il principio guida: livello di garanzia

Ogni numero mostrato appartiene a una di **quattro classi di garanzia**, e la classe è visibile sempre, non in una nota a piè di pagina.

classe	badge	attributi	cosa garantisce
V — vincolato	pieno	anello 1: <code>zona</code> , <code>sex</code> , <code>age</code> , <code>stato_civile</code> , <code>cittadinanza</code> , <code>istruzione</code> , <code>condizione</code> , <code>background</code> , <code>origine_genitori</code>	marginali e incroci del constraint set riprodotti con $MRE \approx 4 \cdot 10^{-4}$
A — allocato	mezzo pieno	anello 3: <code>sezione</code> , <code>age_anni</code> , <code>via</code> , <code>civico</code> , <code>lon/lat</code>	allocazione esatta per sezione (MAE 0,74-1,58 su medie 84-175), ma sotto le assunzioni (8)-(10)
C — condizionato	contorno	<code>paese</code> , <code>area</code>	vincolato ai margini censuari, con struttura geografica secondo il tier (§6). Su Modena, tier 0: nessuna struttura geografica
D — donato	tratteggiato	le 21 AVQ	nessuna informazione geografica . Dipendono solo da <code>sex × macroetà × istruzione4 × regione</code> (assunzione 6)

Conseguenza operativa, e va imposta dall'interfaccia, non lasciata alla prudenza dell'utente:

Qualunque variazione spaziale di una variabile di classe **D** è **interamente compositiva per costruzione**. L'app deve dirlo dove il numero appare, non altrove.

Concretamente: una mappa per sezione di `PUNTI110` non è vietata, ma esce con l'etichetta «*variazione 100% compositiva — nessun segnale areale nei dati*» e con accanto la scomposizione (§4.2). È lo stesso principio di §11.1: la metrica grezza va sempre affiancata alla sua ipotesi nulla.

Un secondo controllo appartiene alla stessa famiglia: **la numerosità efficace**. Le statistiche AVQ di una sottopopolazione di 500 individui non poggiano su 500 osservazioni ma sui donatori distinti che le hanno generate — a Bologna il riuso medio è 84×, quindi 500 individui possono nascere da ~6 donatori. L'app calcola e mostra `n_eff` = donatori distinti, e tutte le bande di incertezza sulle AVQ usano `n_eff`, non `n`. Serve una colonna `donor_id` nel bundle (§3.2): se non è nel `_full`, va aggiunta all'export.

3. Modello dei dati

3.1 Un bundle, non l'albero di lavoro

Il viewer **non legge mai** `~/progetti/gsp/data/comuni/...`. Legge un *bundle* versionato, prodotto da uno script di export. Questo è ciò che rende l'app pubblicabile senza modifiche: in locale e in rete gira sugli stessi byte.

```
bundle/  
  manifest.json          # registro, attributi, etichette, ordini,  
  classi di garanzia  
  qualita.json          # MRE, MAE, correlazioni, copertura AVQ,  
  riuso donatori  
  comuni/  
    017029/  
      pop.parquet        # 198.259 × ~42, categoriche dizionario-  
  codificate  
    sezioni.topojson     # 1.822 poligoni semplificati + P1, ST1,  
  ST16/19, P30-45, ST25-30  
    zone.topojson        # 33 poligoni (dissolve) + nomi verificati  
    reali.json          # tavole reali indicizzate (§3.3)  
    034027/ 037006/ 036023/ ...  
  regioni/  
    emilia_romagna.topojson # confini comunali, per la vista regionale
```

Perché Parquet: le 40 colonne sono quasi tutte categoriche a bassa cardinalità; con dictionary encoding un comune sta in 8–20 MB contro i ~90 MB del CSV. Il motore di query legge **solo le colonne coinvolte nel filtro**, che è la differenza fra 40 ms e 4 s.

Un accorgimento che conta: ordinare le righe per `sezione` prima di scrivere, e tenere row group di ~50k righe. Le statistiche min/max di row group permettono al motore di saltare interi blocchi per qualunque filtro spaziale, che è il filtro più frequente.

3.2 Colonne aggiunte dall'export

colonna	perché
<code>id</code>	stabile, per i permalink all'individuo
<code>donor_id</code>	numerosità efficace delle AVQ (§2). Se non esiste, va tracciata in <code>assign_avq.py</code>
<code>cella_avq</code>	la cella di condizionamento effettivamente usata e il livello di collasso: rende ispezionabile l'assunzione (6)
<code>macroeta</code> , <code>istr4</code>	le variabili su cui <i>davvero</i> girano le AVQ, esplicitate
<code>lon_j</code> , <code>lat_j</code>	coordinate con jitter deterministico entro il civico (§4.3, §7.3)

`cella_avq` merita un commento: è l'unica colonna che rende visibile *quanto* il collasso gerarchico è intervenuto. Una sottopopolazione in cui il 60% degli individui ha ricevuto le AVQ al terzo livello di collasso è molto meno informativa di una in cui tutti sono al livello pieno, e oggi questa differenza è invisibile.

3.3 `reali.json` — l'indice del dato osservato

È la struttura che decide quando il confronto col reale è lecito. Ogni voce dichiara una tavola osservata:

```
{ id: "sez_P1_sesso_eta",  
  fonte: "sezioni_2023",  
  dimensioni: ["sezione", "sesso", "eta"],  
  granularita_spaziale: "sezione",  
  copertura: "totale" }
```

Sorgenti: il file sezioni (`P1`, `ST1`, `ST16/ST19`, `P30-P45`, `P67-P82`, `ST25-ST30`), `targets_K9C.json`, `cs_K9C.json`, le tavole SDMX decodificate, le fonti comunali dei tier 1-3.

Dato un filtro `F` e un attributo da mostrare `A`, l'app cerca una tavola `T` tale che `F` sia esprimibile come selezione sulle dimensioni di `T` e `A ∈ dim(T)`. Se la trova, il riferimento reale c'è. Altrimenti no — e questo è il comportamento corretto, non una mancanza da nascondere.

4. Le viste

Cinque pagine: Città, Esplora, Individuo, Confronta, Metodo.

4.1 Esplora — i filtri

È la pagina centrale. Colonna sinistra: filtri. Corpo: marginali. Destra o sotto: mappa. Tutto in crossfilter.

Filtri di v1 (dodici, non quaranta):

- spaziali: `zona` (multiselezione, o selezione dalla mappa), `sezione` (solo via lazo sulla mappa);
- anello 1: `sezzo`, `eta`, `stato_civile`, `cittadinanza`, `istruzione`, `condizione`, `background`, `origine_genitori`;
- anello 2-3: `paese` (con ricerca; raggruppabile per continente/UE), `area`, `eta_anni` (slider continuo).

Le AVQ non sono filtri in v1. Sono variabili *mostrate*. Il motivo è epistemico prima che tecnico: filtrare su `FIDUCIA` e poi guardare la mappa produce una figura che sembra dire qualcosa sulla geografia della fiducia e non lo dice. In v2 si possono abilitare, dietro un interruttore esplicito che accende un avviso permanente.

Barra di stato del filtro, sempre visibile: `n` selezionati · % della città · `n_eff` donatori AVQ · numero di sezioni toccate · badge rosso se `n < 200` o `n_eff < 30`.

Ogni filtro attivo è una pillola rimovibile; la sequenza dei filtri è la breadcrumb; lo stato completo sta nell'URL (§5).

4.2 Marginali e riferimento — il cuore

Per ogni attributo *mostrabile* un pannello. Griglia di small multiples, ordinabile, con i primi sei fissabili.

Dentro ogni pannello, tre serie sovrapposte:

serie	definizione	interpretazione
sintetico	<code>phat(a)</code>	<code>F</code> `, barre piene
città	<code>phat(a)</code> , contorno grigio	è il valore atteso se <code>A</code> fosse indipendente da <code>F</code> . Lo scarto sintetico-città è l'associazione
reale	dalla tavola <code>T</code> di <code>reali.json</code> , se esiste: marcatore a rombo	la verità censuaria

Il punto sottile: *città* e *indipendenza* sono la stessa serie, perché sotto `A ⊥ F` si ha `P(A|F) = P(A)`. Vale la pena dirlo nell'interfaccia, perché rende gratis la lettura in chiave di ipotesi nulla che serve ovunque (§11.1).

Quando il reale non c'è. Il pannello lo dichiara e — questa è la funzione che vale la pena costruire — **propone il rilassamento osservabile più vicino:**

*Filtro attivo: Fiumicello $\wedge$ donne $\wedge$ laurea. Nessun dato reale per questo incrocio. Dato reale disponibile per: Fiumicello $\wedge$ donne $\rightarrow$ età $\cdot$ Fiumicello $\rightarrow$ istruzione. [mostra]
[mostra]*

Insegna, in uso, quale parte dell'incrocio è misurata e quale è modello. È anche il modo più economico di trasformare la §7 (limiti dichiarati) da appendice a esperienza.

Modalità Δ . Un interruttore trasforma le barre in scarti dalla serie di riferimento, con banda di campionamento $\sqrt{p(1-p)/n}$ — e n_{eff} per le variabili di classe D. Serve a non leggere rumore: su una sottopopolazione di 300 persone una differenza di 3 punti percentuali non è niente.

Scomposizione compositiva (una riga per pannello, solo classi C e D):

AMBIENTE, Fiumicello vs città: $\Delta = -0,07$
spiegato dalla composizione (sesso $\times$ macroetà $\times$ istr4): $-0,07$ (100%)
residuo areale: $0,00$

Per le classi C e D il residuo è **zero per costruzione**, e mostrarlo esplicitamente ogni volta è il modo più efficace di impedire la lettura sbagliata. Per la classe V il residuo è l'errore del modello, $\sim 10^{-4}$, e lo stesso pannello diventa un pannello di validazione.

Tipi di grafico per tipo di variabile: nominali $\rightarrow$ barre orizzontali ordinate per frequenza; ordinali (eta , istruzione , SALUTE , BMI , CPESO) $\rightarrow$ barre in ordine naturale, mai riordinate; 0-10 (PUNTIFI* , VOTOUSL) $\rightarrow$ istogramma + media con IC su n_{eff} ; continue (MH , eta_anni) $\rightarrow$ densità + quantili.

Per la batteria di fiducia serve **un pannello dedicato**: dodici medie su un asse comune 0-10, ordinate, con i riferimenti nazionali AVQ 2024 come tacche fisse (vigili del fuoco 8,10 $\cdot$ forze dell'ordine 6,70 $\cdot$ ASL 6,34 $\cdot$ Comune 5,13 $\cdot$ Regione 4,65) e le fasce di copertura (86-88% / 42-43% / 15-20%) come tratteggio sulle variabili interessate. Sotto, opzionale, la matrice di correlazione con min_periods , che riproduce la struttura a due fattori (servizi vs politica) — è uno dei risultati più belli del dataset e merita una vista propria.

4.3 Mappa

Quattro modalità, un interruttore.

modalità	geometria	quando
Individui	punti ai civici, jitter deterministico	comunicazione, drill-down, «vedere le persone»
Sezioni	1.357-2.224 poligoni	il default analitico: è dove sta il segnale (§10)
Zone	4-33 poligoni	confronto con le fonti comunali, etichette leggibili
Esagoni	H3 res 9/10	quando le sezioni sono troppe o si confrontano città

La metrica di default non è il conteggio. Un conteggio filtrato riproduce la densità di popolazione e non dice nulla: qualunque filtro «illumina» il centro. Le opzioni, in quest'ordine:

1. **quota** — frazione del filtro sul totale locale;
2. **lift** — quota locale / quota comunale, scala divergente centrata su 1;
3. **z** — scarto in unità di deviazione binomiale, che è l'unico modo di non leggere come struttura una sezione da 20 abitanti al 100%;
4. conteggio, ultimo.

Con quota e lift serve **shrinkage empirico-bayesiano** verso la media comunale, oppure almeno lo sbiadimento delle sezioni sotto soglia (~30 individui filtrati). Senza, le mappe di sottopopolazioni piccole sono maculate di falsi estremi. È lo stesso errore contro cui mette in guardia §11.1, nella sua forma cartografica.

Dettagli che vanno gestiti o si vedono subito:

- le **convivenze** (50-630 per comune) stanno al centroide della zona: senza trattamento producono una torre di punti coincidenti. Escluse dai punti per default, con conteggio dichiarato, e simbolo distinto se incluse;
- `indirizzo_fonte = "zona"` (36-278 individui): stesso trattamento;
- più individui per civico: jitter radiale deterministico (funzione dell'`id`), quindi stabile fra sessioni e fra locale e rete) con raggio ~8 m, così il popup resta cliccabile;
- **base cartografica senza chiavi API:** tiles Protomaps `.pmtiles` self-hosted, oppure CARTO Positron. Le tiles self-hosted eliminano l'ultima dipendenza esterna e rendono l'app archiviabile per intero.

Doppio strato per la lettura della sottopopolazione: base grigia (campione della popolazione totale) + strato colorato (il filtro). Serve a distinguere «qui non ce ne sono» da «qui non c'è nessuno».

Selezione spaziale: lazo e clic su poligono aggiungono un filtro spaziale. Da mappa a filtro e ritorno, senza passare dai menu — è il gesto che rende il crossfilter utile.

Campionamento: deck.gl regge centinaia di migliaia di punti, ma il collo di bottiglia è il trasferimento. Tetto a 50k punti con campionamento uniforme e frazione dichiarata in interfaccia; oltre, si passa automaticamente a esagoni.

4.4 Individuo

La carta d'identità di un individuo sintetico. Nasce dal clic su un punto, o dal pulsante *estrai un individuo a caso*.

Quaranta campi, raggruppati per anello, **ciascuno con il proprio badge di garanzia**, e per le AVQ l'indicazione della cella di condizionamento e del livello di collasso. In fondo, la catena di provenienza:

```
Individuo 017029-0084213      [INDIVIDUO SINTETICO – non esiste]
  anello 1  MaxEnt, cella (zona 17029012, F, 35-49, coniugata, ITL, diploma,
occupata)
  anello 2  paese: Italia · AVQ dal donatore #3117, cella (F, 35-49,
istr4=3), livello pieno
  anello 3  sezione 017029000412 · via ... 19A · 45,54 / 10,21
```

Due ragioni per costruirla, oltre alla bellezza. La prima: è lo strumento di debug più efficace che esista, perché rende visibile in un colpo solo tutta la pipeline su un caso. La seconda: è **già il prompt della persona** per il tier 2. Una vista «come l'agente LLM vede questo individuo» — il rendering testuale della persona — collega il viewer al lavoro SimComm senza costruire niente di nuovo, e in una presentazione è il momento in cui il pubblico capisce cosa sono davvero le due tier.

Un banner permanente, non chiudibile, dice che l'individuo non esiste. Vedi §7.3.

4.5 Confronta

Il §8 avverte che il confronto fra città è problematico: risoluzioni diverse su partizioni di taglia diversa. Il design lo prende sul serio con **tre livelli di rigore decrescente e dichiarato**.

Livello A — comune (sicuro). Marginali comunali affiancati, small multiples, una colonna per città. Più la tabella-carta d'identità di §10 (popolazione, zone, sezioni, civici, quota stranieri, tier, donatori, riuso, $(|X|)$). Sempre lecito.

Livello B — distribuzione fra sezioni (il livello giusto). Non si confrontano le zone, si confrontano le **distribuzioni di indicatori a livello di sezione**: quota stranieri, quota laureati, quota over-65. Le sezioni sono unità ISTAT di disegno omogeneo, quindi confrontabili fra città in un modo in cui 4 quartieri di Modena e 33 di Brescia non lo sono. Forme: ECDF sovrapposte o ridgeline. Accanto, la tabella di decomposizione della varianza

di §10 come grafico: il rapporto *tra/dentro* che cresce monotonamente con la taglia media delle zone ($5,9 \times \rightarrow 43,5 \times$) è già un risultato, e in forma grafica diventa **l'argomento visivo** per cui la sezione conta più della zona.

Livello C — zone (con avviso). Mappe affiancate, scale sincronizzate, e un avviso non silenziabile: partizioni non confrontabili, Modena ha 4 zone da 46.000 abitanti.

Un avviso obbligatorio sulle AVQ nel confronto. Parma, Bologna e Modena condividono lo stesso pool di **4.629 donatori emiliani**. Qualunque differenza AVQ fra queste tre città è **integralmente compositiva**: non c'è un solo dato che distingua un bolognese da un modenese, oltre alla loro struttura per sesso, età e istruzione. Brescia attinge a un pool diverso (8.111, Lombardia), quindi un confronto Brescia-Bologna mescola composizione e differenza di pool regionale, che sono cose diverse. Va scritto nella vista, non nella documentazione.

4.6 Metodo e qualità

La pagina che rende l'app difendibile.

- **Validazione:** $MRE(\alpha > 0)$ per comune; MAE per sezione e correlazioni (tabella §10) in forma di scatter osservato-sintetico per sezione, che è più convincente di qualunque numero;
- **Copertura AVQ:** le tre fasce, con la copertura effettiva per variabile e comune, e la nota sul *planned missing*;
- **Riuso dei donatori:** istogramma del riuso, con il tetto strutturale (99,7–100% del pool usato) dichiarato;
- **Le sette assunzioni** di §7 in forma di elenco cliccabile: cliccando l'assunzione (8) si illuminano gli attributi che ne dipendono, ovunque nell'app. È il modo per collegare la teoria all'interfaccia invece di lasciarla in una pagina che nessuno apre;
- **Tier del paese** per comune, con la spiegazione del tier 0 e il numero di §6 (2,08–2,57 volte l'ipotesi nulla a livello di quartiere; 1,01–1,04 a livello di sezione, cioè nulla, correttamente);
- **Provenienza dei nomi di zona:** la verifica su due assi (§3.2), con l'incidente di Bologna raccontato. Un'app che documenta un proprio errore passato è un'app di cui ci si fida.

5. Interazione e stato

Crossfilter completo: cliccare una barra di un marginale aggiunge il filtro corrispondente; disegnare sulla mappa aggiunge un filtro spaziale; entrambi si riflettono nelle pillole e nell'URL.

Stato nell'URL, interamente. Una vista è un link:

Serve a tre cose che valgono da sole lo sforzo: mandare una vista a Tarantino in una riga; citare una figura in un paper con un link riproducibile; e ricostruire in un secondo lo stato in cui hai visto qualcosa di strano.

Esportazioni, ognuna con la provenienza incorporata: PNG/SVG del pannello, CSV della tabella dei marginali (con le colonne di riferimento e i badge di classe), permalink, e — per la mappa — GeoJSON degli aggregati per sezione. Mai l'export dei microdati individuali dall'interfaccia pubblica (§7.3).

6. Grafica

Una direzione, non un tema generico.

Estetica dell'anagrafe. Il riferimento visivo è il registro amministrativo: tabelle a colonne strette, cifre allineate, timbri, carta. Reso in modo contemporaneo, non nostalgico.

Concretamente:

- **tipografia:** un serif con carattere per i titoli e i numeri grandi — un serif «da documento» piuttosto che editoriale; un sans neutro per l'interfaccia; **un monospaziato per tutte le cifre**, con cifre tabulari, così le colonne si allineano da sole. La coerenza dei numeri è il 70% dell'impressione di rigore;
- **colore:** fondo caldo quasi bianco; **un solo accento** per il sintetico; il reale sempre in neutro scuro (rombi, contorni), mai colorato — la gerarchia dev'essere leggibile in bianco e nero. Divergente sobria per il lift; sequenziale monocroma per le quote. Palette sicura per daltonismo, verificata;
- **i badge di garanzia** come piccoli timbri, con forma oltre che colore (pieno / mezzo / contorno / tratteggiato), così funzionano anche stampati;
- **densità alta:** small multiples piccoli e numerosi, nello stile delle tavole statistiche. È più bello e più onesto della singola figura grande, che invita a sovra-interpretare un dato;
- **mappa:** base chiara e desaturata per le coropleti; base scura solo per la vista a punti, dove il fondo scuro fa risaltare la densità;
- **movimento:** transizioni brevi sui cambi di filtro — le barre che si spostano invece di ridisegnarsi rendono percepibile cosa è cambiato. Nient'altro si muove.

Una nota sull'insieme: l'app dev'essere bella nel modo in cui è bella una tavola statistica ben composta, non nel modo in cui è bella una dashboard aziendale. Le seconde invecchiano in due anni e comunicano un'autorevolezza che questi dati non hanno.

7. Architettura e strumenti

7.1 La scelta strutturale

Il vincolo che decide tutto è nella tua frase: «*ora per me ma in futuro andrà in rete fruibile*». Ci sono due strade.

(a) Streamlit / Panel adesso, riscrittura poi. Rapidissimo da mettere in piedi (giorni), tutto Python, ma: non diventa mai «accattivante», non sta su hosting statico, richiede un server sempre acceso, e la versione pubblica sarebbe un secondo progetto.

(b) Applicazione statica dall'inizio. Costruzione dati in Python, interfaccia web, **motore di query nel browser**. In locale gira con un server di sviluppo; in rete è la stessa cosa su GitHub Pages. Nessuna migrazione, nessun server, costo di hosting zero, e i limiti grafici sono solo i tuoi.

Raccomando (b), con una precisazione che ne abbassa molto il costo: non serve scrivere un'applicazione React da zero.

7.2 Strumenti proposti

strato	strumento	nota
costruzione bundle	Python: pandas, geopandas, duckdb, mapshaper	uno script, <code>build_bundle.py {CODICE}</code>
impalcatura	Observable Framework	generatore di siti statici: pagine in Markdown, <i>data loader in Python</i> , build statica, deploy su GitHub Pages. Estetica di default già buona
query	DuckDB-WASM	SQL sul Parquet nel browser, via richieste HTTP a range: scarica solo le colonne e i row group necessari. Aggregazioni su 390k righe in decine di ms
grafici	Observable Plot	grammatica dei grafici concisa; small multiples e faceting con poche righe. Vega-Lite come alternativa
mappa	MapLibre GL + deck.gl	MapLibre per base e coropleti, deck.gl per i punti e gli esagoni H3
geometrie	TopoJSON + mapshaper	2.224 poligoni semplificati stanno in ~300 KB
hosting	GitHub Pages	4 comuni ≈ 60–80 MB. Oltre i ~10 comuni conviene spostare i Parquet su Cloudflare R2 o Hugging Face Datasets con CORS, lasciando l'app su Pages

Observable Framework è la scelta che riduce il codice JavaScript a frammenti nelle pagine, tenendo in Python tutto ciò che è preparazione dati. Se in seguito l'interfaccia crescesse oltre quello che regge, la via d'uscita è Vite + React riusando gli stessi grafici e lo stesso bundle: nessun dato da rifare.

Alternativa onesta se la strada JS non ti convince: Panel + Bokeh (non Streamlit) regge il crossfilter e ha un percorso di esportazione statica via Pyodide — ma con Pyodide DuckDB e deck.gl tornano problematici, e le prestazioni sulla mappa sarebbero il collo di bottiglia. Vale la pena solo se la condizione «tutto in Python» è vincolante.

7.3 Rischi specifici della pubblicazione

Tre, e vanno decisi prima di scrivere codice.

Indirizzi reali. Ogni individuo sintetico sta a un civico ANNCSU esistente. Un residente che cerca il proprio indirizzo trova degli «abitanti» con attributi plausibili. Non c'è divulgazione statistica in senso tecnico — l'individuo non deriva da nessun record reale — ma c'è un problema di percezione e potenzialmente di reputazione, tanto più su Brescia dove il contesto Caffaro è sensibile. Tre livelli possibili, decrescenti in rischio:

1. versione pubblica **senza** `(via)` e `(civico)` nel bundle, punti agganciati al centroide dell'edificio o a una griglia di 25 m; versione interna completa;
2. indirizzo mostrato ma **oscurato al numero** (via sì, civico no);
3. tutto visibile, con banner.

Propendo per (1) per il pubblico e completo per l'uso interno: sono due bundle prodotti dallo stesso script con un interruttore, costo nullo.

Uso improprio dei numeri AVQ. «Nel mio quartiere la fiducia nel Comune è 4,7» è una frase che l'app può involontariamente autorizzare, ed è falsa. Le mitigazioni sono già nel design (§2, §4.2), ma per la versione pubblica considera di **non esporre affatto le AVQ per unità spaziale**, lasciandole solo nei tagli demografici. È una perdita piccola e un rischio molto minore.

Etichettatura. Banner permanente, titolo della pagina, watermark nelle esportazioni PNG, e un campo di provenienza in ogni CSV esportato. Il costo è nullo e protegge il progetto.

8. Estensibilità

Principio: aggiungere una città non deve toccare il codice dell'interfaccia. Tutto ciò che varia sta in `(manifest.json)`, generato da `(G.COMUNI)`.

Il manifest dichiara per ogni comune: codice, nome, slug, regione, pool AVQ regionale, livello ASC in uso e sua etichetta leggibile, numero di zone, tier del paese, attributi presenti, e i numeri di qualità. L'interfaccia si costruisce da lì: se un comune non ha `(zona)` (San Vito dei Normanni), i controlli spaziali di zona semplicemente non compaiono.

Procedura per una città nuova, dopo la §12 del riferimento:

```
build_bundle.py {CODICE}      # parquet + topojson + reali.json + voce di  
manifest  
verifica_bundle.py {CODICE}   # controlli, sotto
```

I controlli del `verifica_bundle` conviene definirli adesso perché sono il presidio contro l'errore che si ripete: totali coincidenti col censimento; nomi di zona presenti e verificati su due assi (§3.2) con un campo `nomi_verificati: [metodo_a, metodo_b]` **obbligatorio** nel manifest; poligoni in numero pari alle zone attese; nessun individuo fuori dal poligono comunale; copertura AVQ nelle fasce attese; `donor_id` presente e riuso coerente.

Emilia-Romagna. Con Parma, Bologna e Modena fatti, i prerequisiti regionali (shapefile R08, indirizzario ANNCSU, `Dati_regionali_2023`, `join_civici_sezioni`) sono già pagati: ogni capoluogo aggiuntivo costa la mezz'ora di §12. Piacenza, Reggio Emilia, Ferrara, Ravenna, Forlì, Cesena, Rimini portano la regione a dieci comuni e ~1,8 milioni di individui — dimensione ancora comodamente gestibile dal browser, un comune alla volta.

Questo suggerisce una **sesta vista, regionale**: la mappa dell'Emilia-Romagna con i comuni coperti, e il confronto di livello B (§4.5) esteso a tutti. Con dieci città la distribuzione fra sezioni degli indicatori diventa un oggetto interessante di per sé, e la decomposizione tra/dentro su dieci partizioni di taglia diversa è una figura che oggi non esiste in nessun paper. Non è un obiettivo di v1, ma la struttura del bundle va predisposta ora perché non costa niente: `regioni/emilia_romagna.topojson` e una chiave `regione` nel manifest.

9. Prestazioni

operazione	atteso
primo caricamento di una città	1,5-4 s (Parquet ~15 MB, TopoJSON ~400 KB)
aggregazione filtrata, 390k righe	20-80 ms
ridisegno dei marginali	< 100 ms
coropleta per sezione, 2.224 poligoni	< 100 ms
punti, 50k	60 fps

Il vincolo vero non è il calcolo ma il trasferimento iniziale. Due mitigazioni: caricare le colonne su richiesta (DuckDB-WASM lo fa da solo se le query sono scritte bene), e precalcolare nel bundle gli aggregati per sezione delle metriche non filtrate, che coprono la vista di ingresso senza toccare il Parquet.

10. Fasi

F0 — decisioni (§11). Nome, stack, destinazione pubblica, semantica del riferimento.

F1 — bundle. `build_bundle.py` su un comune, con `donor_id` e `cella_avq` aggiunti a monte in `assign_avq.py`. Nessuna interfaccia. Verifica dei numeri contro il riferimento v1.6. È la fase che porta più rischio: se `donor_id` non è recuperabile senza rilanciare `assign_avq`, va saputo subito.

F2 — Esplora, senza mappa. Filtri, marginali, riferimento reale, modalità Δ , scomposizione compositiva. Su un comune. È il nucleo: se questa parte funziona ed è leggibile, il resto è lavoro noto.

F3 — mappa. Le quattro modalità, le metriche, shrinkage, selezione spaziale, crossfilter completo.

F4 — le altre città e il manifest. L'estensibilità va provata aggiungendo il quarto comune senza toccare l'interfaccia: se serve toccarla, il design del manifest è sbagliato ed è meglio scoprirlo qui.

F5 — Individuo, Metodo, Confronta.

F6 — grafica in senso proprio, e pubblicazione. La grafica va *dopo*, ma la struttura (tipografia, badge, palette) va decisa in F2, altrimenti si riscrive.

11. Questioni aperte

1. **Semantica del riferimento.** Il design sopra assume tre serie (sintetico / città / reale-se-esiste) con il rilassamento osservabile proposto. È questo che intendevi con «i marginali con quelli reali», o pensavi a qualcosa di più semplice — sintetico contro censimento comunale e basta?
2. **Stack.** Statico web (Observable Framework + DuckDB-WASM) contro Python (Panel/Streamlit). Il primo costa più apprendimento e non richiede riscrittura; il secondo è immediato e va rifatto.
3. **Destinazione.** Pubblico aperto o strumento professionale con link non indicizzati? Decide indirizzi, AVQ per zona, lingua (IT / IT+EN), tono.
4. **AVQ in v1:** solo mostrate, o anche filtrabili?
5. `donor_id`: è recuperabile dai file esistenti, o richiede di rilanciare `assign_avq.py` sui quattro comuni? Da questo dipende metà del design dell'onestà statistica.
6. **Emilia-Romagna:** quanti capoluoghi nei prossimi mesi, e la vista regionale entra in v1 o resta predisposta e vuota?
7. **Sezione o rione per Modena:** se i 37 rioni entrassero in pipeline (§8), la vista Zone di Modena passerebbe da inutile a interessante. Non è una decisione del viewer, ma il viewer è l'argomento più forte per farlo.